Universidad Católica de Murcia (UCAM)

Grado en Ingeniería Informática

Final Practical Assignment - Memoria

Subject: Programación Web

Project: T-Shirt Store - E-commerce Web Application

Field	Value
Subject	Programación Web
Degree	Grado en Ingeniería Informática
Course	2025 / 2026
Convocatoria	Mayo - Julio
Professor	Magdalena Cantabella Sabater
Deliverable	Entrega 2: Server Side (PHP, JavaScript, jQuery, MySQL)

Group Members

Full Name	Email
Marco Antonio Rizzi Meneguetti	marcomeneguetti@gmail.com
Otto Bernado Coutinho	otto.c.lima@gmail.com

Project Briefing

For the final assignment of Programación Web we designed and built a complete, working e-commerce web application: a **T-Shirt Store**. The application reproduces a real sales business process from end to end, covering the four areas required by the assignment: **client logic, product stock, buying/selling, and invoices/reports**.

A customer can browse a catalog served from the database, search products, register and log in, add items to a cart, and complete a checkout that generates a real order and its invoice. On the other side, an administrator has a protected back office to run the **CRUD of products and stock**, **manage users**, and consult **sales reports** and issued invoices.

The project was developed across the two phases defined by the assignment:

- **Phase 1 (client side):** the interface and structure with semantic HTML5 and CSS3, including a brand logo drawn with the **Canvas API**, a navigation menu, a product search bar, a promotions calendar, useful links, and the **Geolocation** HTML5 API to suggest the nearest pickup store.
- **Phase 2 (server side):** the dynamic functionality with **PHP** (written from scratch, with no framework), a **relational MySQL database** accessed through **PDO with prepared statements**, **PHP sessions** for authentication, and **jQuery + AJAX** for the dynamic parts of the interface (cart, search, and the whole admin panel).

Particular attention was paid to **security** (one of the course objectives): passwords are hashed with bcrypt, every query uses prepared statements, forms and AJAX requests are protected with CSRF tokens, protected pages are guarded by the user session and role, and all database-sourced output is escaped before being rendered.

Table of Contents

1. Introduction and Project Description
2. Objectives
3. Technology Stack
4. System Architecture
5. Database Design
6. Phase 1: Client Side
7. Phase 2: Server Side
8. Installation and Execution
9. Demo Accounts
10. Requirements Compliance
11. Conclusion

1. Introduction and Project Description

This project implements a fully functional web application for a real business process, as required by the Programación Web final assignment. The chosen process is an **e-commerce T-Shirt Store** that manages the complete cycle of an online sale: product browsing, customer registration and authentication, stock control, shopping cart and checkout, and the automatic generation of invoices and sales reports.

The application is split into the two deliverables defined by the assignment:

- **Phase 1 (client side):** static design and structure with HTML5 and CSS3.
- **Phase 2 (server side):** dynamic functionality with PHP, JavaScript, jQuery and a relational MySQL database, with session management and security.

2. Objectives

Following the assignment brief, the project pursues these objectives:

- Apply web standards for content development.
- Use client-side and server-side languages and tools.
- Access a database from a web environment.
- Develop a complete web application attending to **accessibility, ergonomics, usability and security**.

The four mandatory business areas are covered:

- **Client logic** - customer registration, login and account.
- **Product stock** - inventory per design, size and color.
- **Buy / Sell / Manage** - shopping cart, checkout and order management.
- **Invoices / Reports** - invoice issuance and sales reporting.

3. Technology Stack

Layer	Technology
Server	PHP 8 (built from scratch, no framework)
Database	MySQL / MariaDB (relational), via XAMPP + phpMyAdmin
DB access	PDO with prepared statements
Client	HTML5, CSS3
Interactivity	JavaScript + jQuery (AJAX)
Web server	Apache (XAMPP)

4. System Architecture

The PHP business logic lives in `src/`, **outside the web root**, so it cannot be requested directly by a browser. Apache serves only the `public/` folder. Pages and AJAX endpoints include the application through a single `bootstrap.php` entry point.

```
prog-web-ucam/
├── database/                # Relational schema and seed data
│   ├── schema.sql           #   CREATE TABLE statements + foreign keys
│   └── seed.sql             #   demo products, stock and accounts
├── src/                     # PHP business logic (NOT web-accessible)
│   ├── bootstrap.php        #   loads config + helpers + models, starts
session
│   ├── config/
│   │   ├── config.php       #   database credentials
│   │   └── database.php     #   PDO connection (singleton)
│   ├── helpers/
│   │   ├── session.php      #   session, CSRF, auth guards
│   │   ├── response.php     #   JSON response helpers
│   │   └── validation.php   #   input validation + output escaping
│   └── models/              #   User, Product, Order, Invoice
└── public/                  # Apache DocumentRoot
    ├── index.php            #   catalog (products from the database)
    ├── login.php register.php logout.php
    ├── account.php          #   customer orders + invoices
    ├── checkout.php         #   order placement
    ├── admin/               #   dashboard, products (CRUD + stock),
users, reports
    ├── api/                 #   JSON endpoints: auth, products, cart,
orders, users
    ├── css/                 #   reset.css, style.css, admin.css
    ├── js/                  #   app.js, auth.js, checkout.js, admin.js
    ├── vendor/              #   jquery.min.js (bundled locally)
    └── assets/              #   product images
```

5. Database Design

The database is **relational** (InnoDB) with foreign keys enforcing referential integrity.

Table	Purpose	Key relationships
`users`	Customers and administrators	-
`products`	Catalog products	-
`product_variants`	Stock per size and color	`product_id` to `products`
`orders`	One order per checkout	`user_id` to `users`
`order_items`	Line items of each order	`order_id` to `orders`, `variant_id` to `product_variants`
`invoices`	One invoice per paid order	`order_id` to `orders`

Relationship summary:

```
users 1---* orders 1---* order_items *---1 product_variants *---1 products
orders 1---1 invoices
```

The `users.role` column (`customer` / `admin`) drives authorization. Stock is decremented inside the same transaction that creates an order and its invoice.

6. Phase 1: Client Side

Requirement	Implementation
Semantic HTML5	`<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>`, `<dialog>`, `<search>`
Company logo with **Canvas**	Hand-drawn hanger logo via the Canvas API (`drawCanvasLogo` in `js/app.js`)
Navigation menu	`<nav>` with catalog / events / pickup links
Web search	Search bar that filters the catalog with jQuery
Events calendar	Month calendar with promotions, navigable by month (`renderCalendar`)
Useful links	"Useful Links" section on the home page
Stylesheets + RESET	`reset.css` (normalize defaults) + `style.css` (layout) + `admin.css`
HTML5 API	**Geolocation** - suggests the nearest pickup store and resolves the delivery address

> The shopping cart used **Web Storage** (`localStorage`) in Phase 1; in Phase 2 it was moved to a server-side **session** cart so orders can be persisted in the database.

7. Phase 2: Server Side

7.1 Sessions and Authentication

PHP sessions track the logged-in user (`$_SESSION['user_id']`, `role`). On login the session ID is regenerated to prevent session fixation. Helpers `require_login()` and `require_admin()` guard protected pages and endpoints.

7.2 Security

- Passwords stored with **bcrypt** (`password_hash` / `password_verify`).
- **PDO prepared statements** on every query (SQL-injection defense).
- **CSRF tokens** issued in the session and validated on every state-changing request.
- Output escaping with `htmlspecialchars` (server) and an `escapeHtml` helper (client).
- Business logic kept outside the web root.

7.3 Client Logic (Users)

Customers register and log in (`register.php`, `login.php`). The account page (`account.php`) lists the customer's orders and invoice numbers.

7.4 Product and Stock Management (CRUD)

The admin panel (`admin/products.php`) provides full **CRUD of contents**: create and delete products, plus add / update / delete stock variants (size, color, quantity). Changes are reflected immediately in the public catalog.

7.5 Sales Flow (Cart, Checkout, Orders)

Items are added to a session-backed cart (`api/cart.php`). At checkout (`checkout.php`) the order is created together with its line items; stock is decremented and an invoice is issued inside a single database **transaction** (`Order::place`).

7.6 Invoices and Reports

Each paid order produces an invoice (`invoices` table). The reports page (`admin/reports.php`) shows total revenue, paid-order count, sales per product and the full list of issued invoices.

7.7 User Administration

The admin panel (`admin/users.php`) lists all users and lets an administrator change a user's role or delete a user (administrators cannot modify their own account from this screen).

7.8 AJAX API and jQuery

Dynamic updates are handled by jQuery against JSON endpoints in `public/api/`:

Endpoint	Responsibility
`auth.php`	register, login, logout
`products.php`	catalog search (public); product/stock CRUD (admin)
`cart.php`	add / remove / list / clear the session cart
`orders.php`	place an order from the cart
`users.php`	list users, change role, delete (admin)

8. Installation and Execution

The application runs on XAMPP (Apache + MySQL).

1. **Start XAMPP** - launch Apache and MySQL from the XAMPP control panel.
2. **Create the database** - open phpMyAdmin (`http://localhost/phpmyadmin`) and import, in order:
   - `database/schema.sql` (creates the `tshirt_store` database and tables)
   - `database/seed.sql` (inserts demo products and accounts)
3. **Configure credentials** - edit `src/config/config.php` if your MySQL user/password differ from the XAMPP default (`root` / empty).

4. **Serve `public/` as the document root** - point an Apache virtual host at `public/`, or place the project under `htdocs/` and browse: `http://localhost/prog-web-ucam/public/index.php`

9. Demo Accounts

Role	Email	Password
Admin	`admin@tshirt.store`	`admin123`
Customer	`cliente@tshirt.store`	`cliente123`

10. Requirements Compliance

Assignment requirement (Entrega 2)	Status	Where
PHP server-side logic	Done	`src/models/`, `public/api/`
JavaScript + jQuery (dynamic UI)	Done	`js/app.js`, `auth.js`, `checkout.js`, `admin.js`
Relational database	Done	`database/schema.sql` (foreign keys)
Use of sessions	Done	`src/helpers/session.php`
CRUD of contents	Done	`admin/products.php` + `api/products.php`
User administration	Done	`admin/users.php` + `api/users.php`
Built from scratch (no framework)	Done	Pure PHP + PDO
Client logic	Done	registration / login / account
Product stock	Done	`product_variants` + admin management
Buy / Sell / Manage	Done	cart to checkout to orders
Invoices / Reports	Done	`invoices` + `admin/reports.php`
Security	Done	bcrypt, prepared statements, CSRF, guards

11. Conclusion

The application fulfils the functional and technical requirements of the assignment for both deliverables: a standards-based HTML5/CSS3 front end with a Canvas logo and the Geolocation API, and a PHP server side with a relational MySQL database, session management, security measures, full CRUD of contents and user administration, all driven dynamically through jQuery and AJAX.